

GitHub Actions: otomatisasi dari commit pertama

**Biar pemeriksaan rutin
dikerjakan otomatis.**

Kenali CI/CD, tulis workflow,
baca hasil, dan perbaiki kegagalan.

Muhammad Asdar · Distritek DevOps

COMMIT



CHECK



RESULT

Setiap perubahan kode perlu diperiksa

Pekerjaan berulang

Ambil kode terbaru.

Siapkan versi runtime.

Jalankan test dan build.

Tantangannya

Langkah bisa terlupa.

Hasil sulit ditelusuri.

Error baru terlihat setelah aplikasi dipakai.

GitHub Actions menjalankan instruksi kita

Otomasi tidak menebak test yang dibutuhkan; kita yang mendefinisikannya.

Kita menulis

Urutan pekerjaan di file YAML.

Tentukan kapan dijalankan dan apa yang diperiksa.

Runner mengerjakan

Mesin mengeksekusi instruksi.

GitHub menampilkan log, status, dan hasilnya.

Git, GitHub, Actions, dan Docker

Git + GitHub

Git mencatat riwayat kode.

GitHub menyimpan repo dan membantu kolaborasi.

GitHub Actions

Mengotomatiskan pekerjaan saat terjadi event pada repo.

Docker

Mengemas dan menjalankan aplikasi.

Dapat digunakan di workflow, tetapi tidak wajib.

Apa yang dimaksud CI dan CD?

CI hijau berarti pemeriksaan yang ditulis lulus; bukan jaminan bebas bug.

CI

Continuous Integration.

Perubahan kode
diperiksa lewat test dan
build otomatis.

Delivery

Hasil yang lulus disiapkan
untuk rilis.

Deploy dapat menunggu
keputusan manusia.

Deployment

Perubahan yang lulus
otomatis diterapkan ke
environment tujuan.

Dari workflow pertama ke CI yang berguna

25 menit

Pengantar dan istilah.
Persiapan repository latihan.

55 menit

Lab 1 Hello: 15 menit.
Lab 2 CI Node: 25 menit.
Lab 3 gagal → pulih: 15 menit.

40 menit

Studi
Docker/React/Compose: 15 menit.
Deployment: 15 menit.
Evaluasi: 10 menit.

Baca workflow dari besar ke kecil

Workflow + event

Workflow: satu file otomatis.

Event: pemicunya, misalnya push atau pull request.

Job + runner

Job: kelompok pekerjaan.

Runner: mesin yang menjalankan job tersebut.

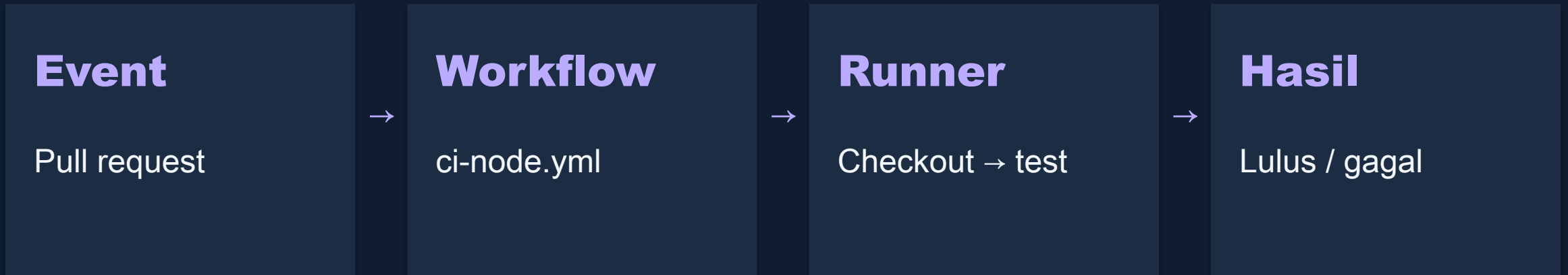
Step + action

Step: satu langkah.

Action: komponen siap pakai yang dipanggil oleh uses.

Dari perubahan kode ke hasil pemeriksaan

Contoh: pull request memicu satu job CI.



Buka run → pilih job → buka step → baca log.

Siapkan tempat belajar yang terpisah

Akun & repo

Akun GitHub dengan akses tulis.

Repository latihan sendiri.

GitHub Actions diizinkan.

File latihan

node-demo/server.js

Dua workflow contoh.

Script smoke-node.mjs.

Batas latihan

Lab awal tanpa secret dan server deploy.

Jangan menyalin workflow deployment aktif.

Indentasi menunjukkan struktur

```
name: Contoh
on:
  workflow_dispatch:
jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Sapa peserta
        run: echo "Halo"
```

jobs berisi job.

steps berisi daftar.

Tanda - memulai item.

Gunakan spasi, bukan tab.

Workflow pertama: Hello Actions

```
name: Lab 1 - Hello Actions
on:
  workflow_dispatch:
permissions:
  contents: read
jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Halo dari GitHub Actions"
```

Simpan sebagai:

`.github/workflows/
hello-actions.yml`

Commit ke default branch.

Jalankan dan baca hasilnya

01 / Pilih

Buka tab Actions.
Pilih Lab 1 - Hello Actions.
Klik Run workflow.

02 / Baca

Pilih run yang baru.
Buka job hello.
Buka step Sapa peserta.

03 / Buktikan

Log berisi pesan Halo.
Job berstatus sukses.
Catat commit yang dijalankan.

uses memakai action; run menjalankan perintah

Potongan di dalam job; bukan workflow lengkap.

```
steps:  
  - uses: actions/checkout@v6  
    with:  
      persist-credentials: false  
  - name: Lihat source  
    run: ls -la
```

Checkout mengambil source ke workspace runner.

Tanpa checkout, jangan menganggap file repo sudah ada.

Kapan pemeriksaan dijalankan?

Trigger dari sample ci-node-pemula.yml.

```
on:  
  workflow_dispatch:  
  pull_request:  
  push:  
    branches: [main]
```

Manual: tombol Run.

PR: usulan perubahan.

Push main: commit masuk ke main.

Job bisa paralel; needs membuat urutan

Tanpa needs

Job A dan B bisa berjalan bersamaan.

Step biasa di satu job berjalan berurutan.

Dengan needs

Job deploy menunggu build berhasil.

needs: build

Data antar-job

Jangan menganggap file lokal tersedia di job lain.

Pindahkan lewat artifact, output, atau registry.

Siapkan Node.js lalu cek sintaks

Potongan job dari ci-node-pemula.yml.

```
steps:
  - uses: actions/checkout@v6
  - uses: actions/setup-node@v6
    with:
      node-version: "22"
      package-manager-cache: false
  - name: Periksa sintaks
    run: node --check node-demo/server.js
```

Source di-checkout.

Versi Node dipilih.

Sintaks dicek sebelum aplikasi dijalankan.

Aplikasi bisa start belum berarti benar

Script menjalankan server sementara dan menghentikannya setelah tes.

```
- name: Uji HTTP dan isi respons  
  run: node samples/github-actions/smoke-node.mjs
```

Test memeriksa:

/health → 200 + ok

/api/info → JSON valid

/tidak-ada → 404

Buktikan CI mengikuti perubahan

Pasang

Salin CI dan script ke path pada panduan.
Commit ke repo latihan.

Jalankan

Run workflow manual.
Buat branch lalu PR.
Amati check di PR.

Hasil

Syntax dan HTTP hijau.
Log menampilkan PASS.
Commit run cocok dengan perubahan.

Sengaja gagal, lalu pulihkan

Rusakkan

Di branch latihan, ganti status /health dari ok ke rusak.

Commit dan push.

Temukan

Syntax tetap lulus.

HTTP test gagal.

Baca expected dan actual pada log.

Perbaiki

Kembalikan nilai ok.

Push commit perbaikan.

Run terbaru harus hijau.

Jalankan perintah Docker yang sudah dikenal

Ilustrasi run step pada runner Ubuntu; bukan workflow lengkap.

```
docker build -t docker-demo:ci ./docker-demo
docker run -d --name web-ci \
  -p 127.0.0.1:8080:80 docker-demo:ci
curl --fail --retry 10 --retry-connrefused \
  --retry-delay 1 http://127.0.0.1:8080
```

Build image.

Jalankan container.

Uji respons HTTP.

Tambahkan pemeriksaan isi.

Pastikan perintah berjalan di folder aplikasi

Potongan steps; working-directory berlaku untuk step run.

```
- uses: actions/setup-node@v6
  with:
    node-version: "22"
    cache: npm
    cache-dependency-path: react-demo/package-lock.json
- run: npm ci
  working-directory: react-demo
- run: npm run build
  working-directory: react-demo
```

Checkout dilakukan sebelumnya.

Lockfile harus cocok.

Build menghasilkan dist/.

Uji aplikasi bersama database

Potongan steps setelah checkout; tambah log saat gagal.

- name: Jalankan stack
run: docker compose up -d --build --wait
working-directory: compose-demo
- name: Periksa layanan
run: curl --fail http://127.0.0.1:3001/health
- name: Cleanup data tes
if: always()
run: docker compose down --volumes
working-directory: compose-demo

Gunakan runner tes sementara.

--volumes menghapus data!

Jangan arahkan ke production.

Cache dan artifact punya tujuan berbeda

Cache

Mempercepat run berikutnya.

Contoh: cache unduhan npm.

Pipeline harus benar saat cache kosong.

Artifact

Menyimpan hasil suatu run.

Contoh: laporan test atau dist.

Bisa diunduh atau dipakai job berikutnya.

Pisahkan konfigurasi dan credential

Dua potongan berbeda, bukan satu workflow utuh.

```
env:  
  APP_ENV: latihan  
  
# Contoh input action yang memerlukan login:  
with:  
  username: ${vars.DOCKER_USERNAME}  
  password: ${secrets.DOCKERHUB_TOKEN}
```

env: nilai di workflow.

vars: konfigurasi GitHub.

secrets: credential.

Jangan cetak secret ke log.

Mulai dengan izin dan waktu yang cukup

Permissions

CI: contents: read.

Tambahkan izin hanya sesuai pekerjaan.

Pin SHA action untuk produksi.

Timeout

Batasi durasi job.

Contoh: 5 menit untuk CI sederhana.

Log harus membantu diagnosis.

Concurrency

CI: run lama boleh dibatalkan.

Deploy: hindari memutus proses penggantian versi.

CI lulus → rilis → deploy → verifikasi

Deployment adalah materi lanjutan setelah CI dipahami.



Health gagal? Pulihkan versi lama dan laporkan deployment gagal.

Sebelum menekan tombol deploy

Tujuan

Registry dan server lab.
Port aplikasi tersedia.
Runtime Docker siap.

Akses

Token registry.
SSH key khusus deploy.
Host fingerprint
tepercaya.

Kontrol

Environment production.
Batas branch + reviewer
bila tersedia.
Health check dan
rencana rollback.

Kenali versi yang sedang berjalan

Tag commit

sha-<commit> membantu penelusuran.

Tag masih dapat ditimpa.

Digest mengikat isi image.

Health check

Cek respons aplikasi setelah deploy.

Proses running saja belum cukup.

Rollback

Simpan referensi versi sebelumnya.

Uji layanan setelah pemulihan.

Pergantian satu container bisa downtime.

Bedakan bahan latihan dan workflow aktif

Sample baru

hello-actions.yml

ci-node-pemula.yml

Di samples/github-actions/; belum aktif.

Sample deploy

deploy-react-demo.yml di samples.

Trigger manual.

Variable
DOCKER_USERNAME.

Workflow aktif

Di .github/workflows/.

Deploy React saat push main atau manual.

Konfigurasi secret berbeda.

Cari step pertama yang gagal

Workflow tak muncul

Cek `.github/workflows/`.
Cek `workflow_dispatch`.
Cek default branch dan akses.

Run merah

Baca log step gagal.
Cek path, sintaks, versi runtime, dan dependency.

Tertunda / skipped

Cek needs dan if.
Cek izin Actions, approval, kapasitas, dan kuota.

Peserta siap lanjut jika bisa...

Menjelaskan

Apa pemicu workflow?

Apa beda job dan step?

Mengapa perlu checkout?

Menunjukkan

Run Hello sukses.

CI otomatis pada PR.

Log assertion yang gagal.

Memperbaiki

Kembalikan CI ke hijau.

Temukan file/workdir salah.

Bedakan CI dengan deploy.

Lanjutkan dengan latihan kecil yang bisa dibuktikan

[praktik-github-actions.md](#)

[samples/github-actions/hello-actions.yml](#) · [ci-node-pemula.yml](#)

docs.github.com/en/actions/get-started/understand-github-actions

github.com/actions/checkout/tree/v6

github.com/actions/setup-node/tree/v6

Panduan resmi workflow manual dan artifact: lihat catatan slide.